Erasure-Compliant, Differentially Private Distinct Counting under Continual Observation

Anonymous Author(s)

Abstract

Distinct-count analytics such as Daily Active Users (DAU) and Monthly Active Users (MAU) are metrics for product telemetry, yet they conflict with privacy regulations like GDPR Article 17 ("Right to be Forgotten"). We present a **reference architecture** for erasure-compliant, differentially private distinct counting under continual observation. Our system synthesizes three key components: (i) a **tiered salt management strategy** for cryptographic forward secrecy; (ii) a **state backend with eager tombstone propagation** to mitigate application-level timing leakage; and (iii) **sensitivity-bounded Rényi Differential Privacy (RDP)** composition for tight budget accounting.

We address the "Privacy-Performance Gap" by implementing a vectorized, log-structured storage engine. Experimental evaluation on a single-node commodity system, reproducible from the committed reference implementation, demonstrates that our architecture scales to **1 million users** with ≈**108k events/sec** ingest (≈11.75× over the prior per-event SQLite ledger) on an exact 64-bit Roaring-bitmap backend. For Monthly Active Users, we report the query latency honestly as two numbers rather than one: a genuinely new 30-day window costs ≈2.7 s (p50) at 1M users—the true cost of an exact union of 30 daily bitmaps—while a repeated query of the *same* window, the realistic dashboard-polling access pattern, is served in ≈1.5 ms (p99) via version-fingerprinted memoization of the pre-noise union, exact rather than approximate. We prove that the system satisfies (ε, δ)-differential privacy with constant sensitivity $\Delta = 1$, ensuring that compliance with erasure requests does not degrade the privacy guarantees of remaining users.

Keywords

Differential Privacy, Distinct Counting, GDPR, Streaming Algorithms, Privacy-Preserving Analytics

1 Introduction

Distinct-count metrics such as Daily Active Users (DAU) and Monthly Active Users (MAU) are among the most widely reported statistics in industry: they anchor product decisions, advertising billing, capacity planning, and public financial disclosures across mobile analytics, IoT monitoring, advertising, and retail. Because they are computed continuously over the activity of identifiable individuals, they are also personal-data processing in the eyes of modern privacy regulation—and the two obligations that regulation imposes pull in opposite directions.

On one side, releasing a fresh count every day is a textbook instance of *continual observation*: without careful accounting, the cumulative privacy loss of daily releases grows without bound, and an observer correlating releases can learn far more about an individual than any single number reveals. On the other side, the "right to be forgotten" demands *retroactive* removal: a user who requests erasure must disappear not only from tomorrow's count but from every aggregate the system will ever release again—including re-releases of historical windows. The first obligation is solved by adding calibrated noise and accounting for it; the second requires the system's *pre-noise state* to be mutable in a way that popular streaming summaries (append-only sketches such as HyperLogLog) simply are not. Systems deployed in practice typically satisfy one obligation or the other; this paper is about satisfying both at once, at production throughput.

These analytics are subject to two intersecting requirements. First, *privacy regulations* such as GDPR Article 17 [30] and CCPA [31] grant users a "right to be forgotten," requiring that user data be retroactively removed from all aggregates upon request. This requirement has spurred research in *machine unlearning* [32–34], though most existing approaches target ML models rather than streaming analytics. Second, *differential privacy* (DP) has emerged as the standard for meaningful privacy guarantees in statistical releases, bounding the influence any single user can have on published outputs.

Supporting both requirements simultaneously presents significant technical challenges:

(1) **Turnstile streams:** Deletions transforms monotone stream (insertions only) into a turnstile stream where user presence toggles between "active" and "deleted."
(2) **Continual release:** Publishing daily metrics requires careful composition analysis to track cumulative privacy loss over time.
(3) **Retroactive deletion:** GDPR requires removing deleted users from *historical* aggregates, not just future counts.

Recent theoretical work by Jain et al. [1] addresses distinct counting with deletions via the *flippancy* parameter w—the maximum number of times a user's status can flip—achieving error $\tilde{O}(\sqrt{w})$ at item-level DP and proving $\Omega(\min(T^{1/3}/\varepsilon^{2/3}, T))$ lower bounds for unbounded flippancy. Our work bridges this theory to practice by implementing a complete system that achieves these guarantees through daily coalescing (bounding w at a practical level) combined with explicit tombstone propagation for deletion handling.

Contributions. We make the following contributions:

(1) We present a **reference architecture** for erasure-compliant, differentially private distinct counting under continual observation, spanning edge pseudonymization, a deletion-native state tier, and a DP release tier with unified Rényi-DP budget accounting, together with a working single-node reference implementation.

This work is licensed under the Creative Commons Attribution 4.0 International License. To view a copy of this license visit https://creativecommons.org/licenses/by/4.0/ or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.
Proceedings on Privacy Enhancing Technologies YYYY(X), 1–17
© YYYY Copyright held by the owner/author(s).
https://doi.org/XXXXXXX.XXXXXXX

(2) We **formalize the erasure guarantee**: a post-erasure release-semantics definition (Definition 4), an explicit *history-independence* requirement that a state backend must satisfy for the guarantee to hold (Definition 5), and an epoch-overlap protocol that preserves the guarantee across salt rotations (Section 4.2).
(3) We present a **tombstone-based deletion mechanism** that eagerly updates hot state and propagates deletions to all affected historical state via compaction replay, keeping query latency independent of deletion volume and thereby mitigating application-level timing leakage.
(4) We give a **privacy analysis** covering each release (sensitivity-1 mechanisms under daily coalescing), the deployment horizon (a ledger-enforced total-loss bound), and the interaction of deletions with the accounting.
(5) We report a **comprehensive, reproducible evaluation** at up to 1,000,000 users on the committed reference implementation: relative error below 0.3% at $\varepsilon = 1.0$; ingestion at ≈108,000 events/second via a binary-log engine (≈11.75× over the prior per-event SQLite ledger); DAU query latency that stays in the low milliseconds; and MAU query latency reported honestly as a fresh-window cost (≈2.7 s at 1M users, the true price of an exact 64-bit union) versus a cached-repeat-query cost (≈1.5 ms at every scale tested, via version-fingerprinted memoization)—the realistic access pattern for dashboards that poll the same window repeatedly.
(6) We include two **design-exercise case studies**—a canonical product-telemetry deployment and a high-sensitivity physical-space (CCTV) variant—with an explicit account of the simulation methodology behind each.

Positioning. Table 1 summarizes the landscape: no prior system simultaneously achieves central DP, native deletion support, and continual release with RDP composition. Our architecture is the first to unify all three properties.

Why Existing Approaches Fail. Current privacy-preserving analytics systems exhibit several limitations. Firstly, *append-only sketches*, such as HyperLogLog, lack the capability to support element deletion, necessitating comprehensive rebuilds that do not scale effectively. Secondly, *standard telemetry pipelines* are rendered ineffective under the GDPR's deletion mandates, as they do not accommodate the retroactive removal of data from historical aggregates. Finally, the paradigm of *continual observation* within the framework of differential privacy necessitates meticulous composition accounting; however, naive implementations frequently fail to achieve this, resulting in unbounded privacy loss.

2 Background and Related Work

2.1 Differential Privacy

Differential privacy [2, 3] establishes a rigorous mathematical framework for assessing privacy loss associated with the dissemination of statistical information. This framework allows for a quantitative evaluation of the trade-off between data utility and the protection of individual privacy in statistical releases.

Definition 1 ((ε, δ)-Differential Privacy). *Randomized mechanism $\mathcal{M} : \mathcal{D} \to \mathcal{R}$ satisfies (ε, δ)-differential privacy if for all neighboring databases $D, D' \in \mathcal{D}$ differing in one individual's data, and all measurable sets $S \subseteq \mathcal{R}$:*

$$\Pr[\mathcal{M}(D) \in S] \leq e^{\varepsilon} \Pr[\mathcal{M}(D') \in S] + \delta$$

The parameter ε governs the tradeoff between privacy and utility in differential privacy frameworks. Specifically, a smaller value of ε corresponds to enhanced privacy guarantees; however, this increase in privacy is accompanied by the necessity for greater noise to be introduced into the data. Typical production systems use $\varepsilon \in [0.1, 10]$, with $\varepsilon \leq 1$ considered "strong" privacy. The δ parameter represents the probability of a catastrophic privacy failure and is typically set to $\delta \ll 1/n$ where n is the database size.

For numeric queries with sensitivity Δ (the maximum change from adding/removing one individual), the *Laplace mechanism* adds noise $\eta \sim \text{Lap}(\Delta/\varepsilon)$ to achieve ε-DP [2], while the *Gaussian mechanism* adds $\eta \sim \mathcal{N}(0, \sigma^2)$ with $\sigma = \Delta\sqrt{2\ln(1.25/\delta)}/\varepsilon$ to achieve (ε, δ)-DP [3, 10]. The choice between mechanisms depends on the query structure: Laplace provides pure ε-DP suitable for single releases, while Gaussian enables tighter composition bounds via RDP for repeated queries.

Pseudonymization vs. Anonymization. Our system performs *pseudonymization* as defined by GDPR Recital 26 [30]: salted hashes replace direct identifiers, but re-identification remains possible for authorized parties (the KMS) holding the epoch salt. This is strictly weaker than full anonymization, and the processed data therefore remains within the scope of GDPR. Differential privacy provides an orthogonal, quantifiable guarantee on the *released statistics*, bounding the information any observer can infer about any individual from published outputs.

2.2 Composition and Rényi DP

When releasing multiple statistics, privacy loss accumulates. Basic composition sums ε values [8], but this can be pessimistic. *Rényi Differential Privacy* (RDP) [6] provides tighter bounds:

Definition 2 (Rényi Differential Privacy). *A mechanism $\mathcal{M}$ satisfies (α, ρ)-RDP if for all neighboring D, D':*

$$D_\alpha(\mathcal{M}(D) \| \mathcal{M}(D')) \leq \rho$$

where D_α is the Rényi divergence of order $\alpha > 1$.

RDP composes linearly: k applications of an (α, ρ)-RDP mechanism yield $(\alpha, k\rho)$-RDP. Converting back to (ε, δ)-DP via $\varepsilon = \rho + \frac{\ln(1/\delta)}{\alpha - 1}$ often yields tighter bounds than basic composition. For 30 daily MAU releases at $\varepsilon = 0.5$ and $\delta = 10^{-6}$, our implementation's RDP conversion gives $\varepsilon_{\text{total}} \approx 3.04$ versus 15.0 for naive summation (Section 5).

2.3 Distinct Counting Sketches

Streaming algorithms designed for the purpose of distinct counting facilitate a trade-off between exact computation and sublinear memory utilization [21, 22].

HyperLogLog (HLL/HLL++) [16, 20] employs statistics derived from bit-patterns to provide an estimation of cardinality, utilizing $O(m)$ memory with a relative standard error of approximately

$1.04/\sqrt{m}$. However, it is important to note that HLL operates under an *append-only* paradigm—registers are constrained to storing maximum values and do not support decrement operations.

KMV (K Minimum Values) sketches, on the other hand, maintain the k smallest hash values, which enables cardinality estimation through the formula $(k-1)/h_k$, where h_k represents the k-th smallest hash value. A critical advantage of KMV lies in its support for explicit element removal, as it allows for the deletion of hash values [17].

Theta Sketches [18] extend the capabilities of KMV by incorporating native set operations, including A-not-B (set difference), thereby rendering them particularly suitable for scenarios involving deletion. In our reference architecture, KMV and Theta sketches serve as viable probabilistic backends; however, our evaluation instantiates **Roaring Bitmaps** (exact compressed sets) to isolate DP noise from sketch approximation error (see Section 6).

2.4 Turnstile Distinct Counting with DP

In their seminal work, Jain et al. [1] investigate distinct counting under the framework of differential privacy (DP) within turnstile data streams, specifically focusing on scenarios involving continual observation. They establish an operational efficiency characterized by an $O(1)$ query cost for Daily Active Users (DAU) and a bounded union overhead for Monthly Active Users (MAU). Central to their analysis is the concept of *flippancy* w, which denotes the maximum frequency with which any user's presence indicator can oscillate. Their findings include the following significant results:

- Event-level DP faces $\Omega(\min(T^{1/3}/\varepsilon^{2/3}, T))$ lower bounds where T is the time horizon
- Item-level DP with flippancy bound achieves error $\tilde{O}(\sqrt{w})$

Flippancy serves as a metric for the "volatility" inherent in a user's participation pattern. For instance, a user whose engagement is limited to a single instance would exhibit $w = 1$, while a user who alternates states daily would possess $w = T$. The insight is that constraining flippancy facilitates the achievement of sublinear error relative to the time horizon.

Our proposed daily coalescing method effectively limits the flippancy of individual users by ensuring that each user contributes at most one event per day, thereby attaining practical item-level differential privacy. Formally, if user states are exclusively assessed at the boundaries of each day, the flippancy for any user is constrained by $w \leq T$, with T denoting the total number of days. However, empirical observations suggest that typical user behaviors result in $w \ll T$, given that most users do not frequently alternate between active and inactive states.

2.5 Production Privacy Systems

Google's RAPPOR [25] pioneered local DP for telemetry collection, enabling Chrome usage statistics without central data access. Apple's implementation [24] similarly uses local DP for emoji and keyboard telemetry. However, both systems are designed for frequency estimation rather than distinct counting.

Aggregation Systems. Google's Prochlo [26] combines shuffling with DP for improved privacy amplification. Prio [27] uses secret sharing for privacy-preserving aggregation. Neither system addresses retroactive deletions, as their security models assume append-only data streams.

Streaming DP. Dwork et al. [11] introduced the continual observation model, requiring privacy across unbounded releases. Chan et al. [12] provided tree-aggregation for accuracy. Our work extends this by incorporating deletions via tombstones.

Gap Analysis. While industrial systems such as Google's RAPPOR [25] and Apple's Differential Privacy [24] operate at massive scale, they rely on **Local DP** (randomized response) or **Append-Only** sketches (HLL/CMS). These structures lack native support for the item-level deletions required by GDPR Article 17. To support deletion, these systems would require full data replay, incurring prohibitive computational costs. Our architecture fills this gap by introducing a deletion-native state backend (pluggable KMV or Roaring Bitmaps) that maintains stable, low-latency query performance even under high churn (Table 1).

Table 1: Comparison with real-world privacy systems

System	Privacy	Deletions	Continual	Data Structure
LinkedIn [23]	Central DP	× (Append)	✓	HLL
Apple [24]	Local DP	N/A	×	Count Mean
RAPPOR [25]	Local DP	×	×	Bloom Filter
VideoDP [36]	Central DP	×	✓	Sampled
Our System	**Central DP**	**✓(Tombstone)**	**✓(RDP)**	**Pluggable Backend**

3 Problem Formulation and Threat Model

3.1 System Model

We consider a stream of activity events from users interacting with a service. For day d, let $S_d \subseteq \mathcal{U}$ denote the set of users active on that day, where $\mathcal{U}$ is the universe of possible user identifiers. We define:

$$\mathrm{DAU}(d) = |S_d| \tag{1}$$

$$\mathrm{MAU}_W(d) = \left| \bigcup_{i=d-W+1}^{d} S_i \right| \tag{2}$$

where W is the window size (typically 30). Note that $\mathrm{MAU}_W(d) \geq \mathrm{DAU}(i)$ for all $i \in [d-W+1, d]$, as the monthly metric aggregates all daily users into a set. This nesting property is crucial for sensitivity: a user contributes at most 1 to the MAU count regardless of active days within the window.

3.2 Threat Model

Parties and trust. The system comprises *edge devices* (untrusted; they pseudonymize and immediately discard raw identifiers), an *aggregation tier* (gateway and cloud), and an isolated *key-management service* (KMS). We are precise about the aggregation tier's role, since "trusted" and "honest-but-curious" name different properties: the aggregation tier is trusted for *correctness*—it faithfully executes the protocol (daily coalescing, tombstone application, noise addition, ledger enforcement)—but is *curious with respect to what it observes*: our guarantees never assume it forgets data or metadata that passes through it. Differential privacy protects individuals in the *released outputs* against any observer, including the aggregation

tier's operators reading dashboards; pseudonymization limits what the ingestion path itself retains. We explicitly do not defend against a *malicious* aggregator that deviates from the protocol (e.g., releasing unnoised counts); hardening against that adversary requires secure computation or attested execution and is out of scope.

Privacy goal. We target *user-level* (item-level) differential privacy: adding or removing *all* events of a single user has bounded influence on all outputs. This is stronger than event-level DP, which protects single events rather than entire user histories.

Definition 3 (Neighboring Databases). *Two event streams D and D' are neighbors if they differ in all events from exactly one user u—that is, $D' = D \setminus \{(u, \cdot)\}$ or vice versa.*

An adversary observing all published statistics—and able to inject events or deletion requests to probe the system—cannot determine with high confidence whether a specific user participated, even knowing the participation of all other users (the "strong adversary" model).

The KMS contract. The KMS is the only party holding long-term *epoch salt seeds*. Its interface is deliberately narrow: given a *verified* erasure request for user u, it derives the pseudonymous tokens h_u for the affected epochs and issues signed tombstone tokens to the gateway; raw salts never leave the KMS, and neither the gateway nor the cloud tier can derive historical pseudonyms on their own. What is hidden, from whom: salt seeds are hidden from the aggregation tier, edge devices, and external adversaries; the KMS learns only which (verified) users requested erasure. Compromise of the KMS is the strongest failure in our model: it enables pseudonym inversion for the affected epochs, degrading pseudonymization—though released statistics remain protected by DP, whose guarantees do not depend on salt secrecy.

What the gateway sees. The gateway necessarily observes transport metadata: which edge device submitted which pseudonym, and when. Pseudonymization protects identifier *content*, not traffic metadata; deployments requiring sender anonymity toward the gateway would need batching or mixing in front of ingestion, which we do not provide. We state this explicitly rather than implying the gateway is blind.

Erasure requests are control-plane data. The *fact and timing* of an erasure request are visible to the aggregation tier (which must act on them) and are not themselves DP-protected. What DP protects is the user's presence in every *released statistic*: post-erasure releases are fresh, ledger-charged DP releases over the updated state (Definition 4), so an external observer of outputs cannot confidently detect any individual's participation—or erasure—from the published numbers.

Summary of guarantees.

- ✓ **Guaranteed:** (ε, δ)-DP for all released DAU/MAU counts against any output observer; GDPR Article 17 compliance via retroactive tombstone propagation (Definition 4); total privacy loss enforced by an RDP ledger over the deployment horizon (Section 5).
- × **Not covered:** network-level traffic analysis; OS jitter and microarchitectural side channels; physical server compromise; sender anonymity toward the gateway; a malicious (protocol-deviating) aggregator. Application-level query-latency leakage is partially mitigated by eager tombstone application (Section 4).

The DP guarantees treat hash functions as random oracles. Pseudonymization additionally assumes epoch salts remain confidential; salt compromise permits pseudonym inversion but does not void the DP guarantees on released statistics.

3.3 Connection to Flippancy Theory

Jain et al. [1] characterize the hardness of DP distinct counting in turnstile streams via the *flippancy* parameter w: the maximum number of state changes (active ↔ inactive) a user can perform, proving $\Omega(\min(T^{1/3}/\varepsilon^{2/3}, T))$ worst-case lower bounds when flippancy is unbounded. Our daily coalescing bounds each user's contribution to one event per day, so w is bounded by the number of days a user actually toggles—empirically far below the horizon T for realistic engagement patterns. Their matching sublinear upper bound also signals that correlated-noise mechanisms (e.g., tree aggregation) can beat per-release noise in the long-horizon regime; we return to this in Section 11.

3.4 Deletion Requirements

Upon receipt of a deletion request pertaining to user u, the system is mandated to fulfill the following requirements via *tombstones*—signed deletion markers that record the removal of a specific hashed identifier from a specific day's state:

(1) Eliminate the presence of user u from all historical day sets S_d in which u was included.
(2) Guarantee that all subsequent Daily Active User (DAU) and Monthly Active User (MAU) queries accurately reflect the removal of u.
(3) Maintain differential privacy (DP) guarantees post-deletion.

We formalize this guarantee as follows:

Definition 4 (Post-Erasure Release Semantics). *For any user u and any time t after a verified erasure request, every subsequent release satisfies:*

$$\mathit{Release}_t \overset{d}{=} \mathcal{M}(D \setminus \{u\})$$

where $\mathcal{M}$ is the DP mechanism instantiated with fresh randomness and the release is charged to the RDP ledger. The pre-noise state is updated deterministically to reflect $D \setminus \{u\}$.

The fresh randomness in each post-erasure release limits correlation across releases: an adversary observing outputs before and after a deletion cannot cancel noise to recover the exact effect of the erasure. Outputs released *prior* to the deletion cannot be retroactively altered, as they are superseded by subsequent releases. This represents a constraint inherent in any DP framework: once noise has been added and disseminated, the resulting output remains immutable. Nevertheless, future queries will appropriately exclude deleted users, and the RDP ledger ensures that the cumulative privacy loss across all releases—including re-releases triggered by deletions—is properly accounted for.

Definition 4 constrains the *pre-noise state*: after erasure of u, the state a release is computed from must equal the state that would exist had u never been ingested. Whether a given backend can satisfy this is a property of the backend itself, which we make explicit:

Definition 5 (History-Independent Deletion). *A state backend with operations add and remove supports* history-independent deletion *if, for every input multiset D and user token h_u, the backend state after ingesting D and then executing remove(h_u) is identically distributed to the state after ingesting $D \setminus \{h_u\}$ directly.*

History independence is what makes Definition 4 achievable: if deletion merely *masks* an element while leaving a trace of its former presence in the structure (as in append-only sketches, or in sketches whose retained samples depend on insertion order relative to the deleted element), then post-erasure releases are not distributed as $\mathcal{M}(D \setminus \{u\})$ and the guarantee fails. Exact set representations (e.g., compressed bitmaps) satisfy the property trivially. KMV and Theta sketches satisfy it only under conditions—deletion must fully restore the sketch to the state it would have reached without the deleted element, including re-admitting displaced candidates—which standard implementations meet by retaining sufficient auxiliary state. Our architecture therefore treats Definition 5 as an admission requirement on pluggable backends (Section 4.4), and our evaluation instantiates an exact backend for which the property holds unconditionally. Section 7.3 reports a direct empirical test of this definition against the repository's own backend implementations, rather than leaving it as an unverified assertion.

4 System Design

4.1 Architecture Overview

As depicted in Figure 1, our proposed architecture is structured into three distinct tiers, facilitating a separation of concerns. Edge devices are responsible for the pseudonymization process and do not retain raw identifiers, while the gateway maintains the state of sketches and manages the deletion of data. The cloud aggregator is charged with the application of differential privacy noise and the enforcement of budgetary constraints. This architectural design adheres to the principle of *data minimization*, ensuring that each tier retains only the data necessary for its designated functions, thus significantly reducing the overall attack surface.

4.2 Pseudonymization

To break long-term linkability while maintaining consistency within MAU windows, we use epoch-stable salted hashing:

$$\text{user_key} = H(\text{raw_id} \| \text{salt}_{\text{epoch}})$$

where $\text{salt}_{\text{epoch}}$ rotates every 365 days. This ensures:

- Same user maps to same key within any 30-day window
- Cross-epoch linkage is cryptographically hard
- Raw identifiers are immediately discarded

Cross-Epoch Handling. Salt rotation raises an immediate correctness question: a query window that straddles a rotation boundary sees the same user under two different pseudonyms, so a naive union would double-count every user active on both sides. We resolve this with an explicit *overlap protocol*. Let E be the epoch length, $W_{\max}$ the largest supported query window (default $E = 365$ days, $W_{\max} = 62$), with $E > W_{\max}$ required. During the *overlap period*—the first $W_{\max} - 1$ days of each new epoch—the Gateway holds derivation state for both the retiring salt s_i and the active salt s_{i+1}. Each arriving raw identifier is (at the Gateway, before discarding) mapped under *both* salts, and the pair $\big(H(x \,\|\, s_i),\, H(x \,\|\, s_{i+1})\big)$ is recorded in a transient *bridge table* for the overlap period only. A window query crossing the boundary unions old-epoch day sets and new-epoch day sets after rewriting old-epoch tokens through the bridge, so each user contributes one identity.

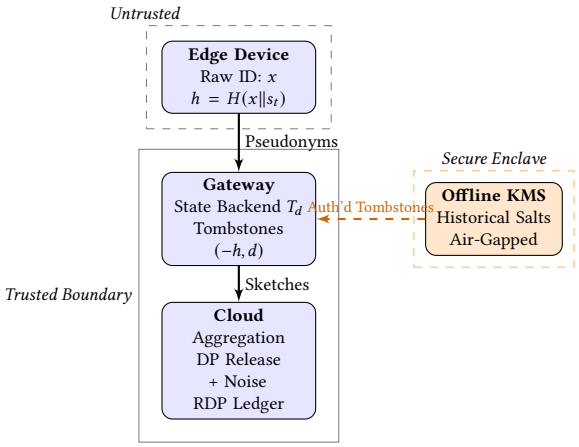


Figure 1: Reference architecture with Privacy Officer KMS. Edge devices pseudonymize with epoch salts (discarded after use). The Offline KMS retains historical salts in an air-gapped enclave, enabling authorized retroactive deletions without exposing keys to the ingestion tier.

Proposition 6 (Cross-Epoch Window Correctness). *Under the overlap protocol, for any window of length $W \le W_{\max}$ ending at day d, the pre-noise count equals $|\bigcup_{i=d-W+1}^{d} S_i|$ computed over* true *user identities, and the per-user sensitivity of the released count remains $\Delta = 1$.*

Proof sketch. Any window crossing the boundary lies entirely within $[\, d_{\text{rot}} - W_{\max} + 1,\ d_{\text{rot}} + W_{\max} - 1 \,]$, where d_{rot} is the rotation day. Every user active on both sides within such a window is, by construction, observed during the overlap period and thus has a bridge entry; rewriting makes their two tokens equal, so the union counts them once. A user active on only one side contributes one token. Hence the union cardinality equals the count over true identities, and each user still changes it by at most 1. □

The bridge table is itself pseudonymous (it maps hash to hash, never storing raw identifiers), lives only for the overlap period, and is covered by the same tombstone flow: an erasure during overlap removes the user's bridge entry along with their day-set memberships. Deployments whose maximum window respects $W_{\max}$ never pay any client-side cost; rotation remains invisible to edge devices.

To resolve the tension between Forward Secrecy (rotating salts) and Retroactive Deletion (GDPR), we introduce a logical **Privacy Officer KMS (Key Management System)**. This isolated, restricted component retains historical salt seeds strictly for deletion authorization. Edge nodes and Gateways discard old salts to prevent local compromise. When a deletion request is verified, the KMS re-derives the specific historical hashes required for tombstones and pushes them to the Gateway via a secure synchronization process, ensuring that erasure is mathematically possible without exposing long-term keys to the ingestion tier.

Figure 2 illustrates the relationship between salt epochs, MAU windows, and deletion requests.

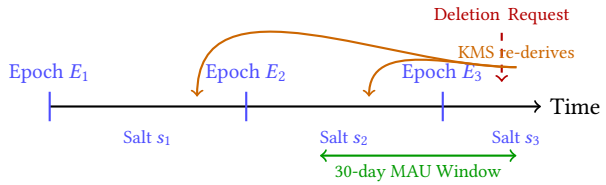


Figure 2: Epoch timeline showing salt rotation and retroactive deletion (schematic; *not to scale*—each epoch spans 365 days while the query window spans 30). When a deletion request arrives, the KMS re-derives historical hashes using retained salt seeds (s_1, s_2) to generate tombstones for all affected days within the MAU window.

4.3 State Backend with Tombstones

For each day d, we maintain a state backend T_d storing user presence information. The architecture supports multiple backends—probabilistic sketches (KMV, Theta) or exact compressed sets (Roaring Bitmaps)—with a common deletion interface. For MAU, we compute:

$$T_{\text{MAU}}(d) = \text{Union}(T_{d-W+1}, \ldots, T_d)$$

Deletion via Tombstones. When user u requests deletion:

(1) **Identify Scope:** The KMS derives hash tokens h_u for the user across the requested erasure window (one token per affected epoch).
(2) **Log Tombstones:** The system appends deletion records $(-h_u, d)$ to the append-only log for every affected day d.
(3) **Eager State Update (hot state):** The Gateway *immediately* invokes `backend.remove(`h_u`)` on all *resident* day states—the current day and any day backends held in memory for active query windows. This operation is atomic and independent of query traffic, so the latency of subsequent queries does not depend on deletion volume.
(4) **Tombstone Replay (historical state):** Deletion is *not* limited to the current day. Asynchronous compaction workers replay logged tombstones against every affected historical day backend as segments are compacted, invoking `backend.remove(`h_u`)` on each stored T_d with $u \in S_d$. A tombstone is considered *settled* only when every affected backend has confirmed removal.
(5) **Purge:** Once settled, the tombstone record itself is purged from the log.

Why tombstones are compatible with erasure. A natural objection—raised in regulatory terms as "why is it acceptable to retain a deletion record about a user who wishes to be forgotten?"—deserves a direct answer. Tombstones are *transient control-plane artifacts*, not retained user data: they contain only a salted hash and a day index, never plaintext identifiers; they exist precisely to *execute* the erasure across historical state; and they are purged once settled. After settlement, the system's entire state is, by Definitions 4 and 5, equivalent to one in which the user was never ingested: the user is not "toggled to deleted"—the toggle is the mechanism in flight, and removal is the end state. This transient-processing pattern is consistent with GDPR Article 5(1)(c) (data minimization) and with Article 17's own contemplation that a controller must process the request itself in order to honor it.

This **Eager Tombstone Application** strategy ensures that query latency is **independent of deletion volume**: a deletion never makes a subsequent query of the same window slower or faster than an equivalent query with no pending deletions. For DAU queries, cardinality lookup executes in $O(1)$ time. For MAU queries spanning 30 days, latency is instead dominated by whether the 30-day union has been computed before: a genuinely new window costs multiple seconds at 1M-user scale (an exact 64-bit Roaring union, measured at ≈2.7 s p50 / ≈4.5 s p99, Section 7.8), while a repeat query of an already-computed window returns the memoized union in ≈0.6 ms p50 / ≈1.5 ms p99 (Section 4, "Window-Union Caching with Versioning"). Deletion volume affects neither number: a deletion invalidates the cached union for the affected window (forcing one honest recomputation), but does not otherwise change query cost, preventing adversaries from inferring user presence via timing attacks.

4.4 Backend Selection

Table 2 compares potential state backends. Our architecture supports KMV for approximate counting with deletion, Theta for native set operations, and Roaring Bitmaps for exact counting. HLL is unsuitable on two grounds: it lacks item-level deletion, and—per Definition 5—no masking workaround can make an append-only register structure history-independent under deletion. Admission to the pluggable backend factory requires the history-independence property.

Data minimization vs. exactness. Backend choice is not only a performance and accuracy decision; it changes the *disclosure surface if the gateway is compromised.* An exact backend (Roaring bitmaps) stores the full membership set of hashed identifiers per day: an attacker who seizes gateway state learns, for every stored day, exactly which pseudonyms were active—re-linkable to individuals if the attacker can also query the current epoch's hash (e.g., by submitting candidate identifiers). A KMV or Theta sketch retains only a bounded sample of hash values, so the same compromise discloses membership information about a bounded subset of users, at the cost of approximation error in counts. Deployments therefore face a genuine minimization–functionality trade-off, which our pluggable factory exposes as a deployment-time choice: compact sketches

for stronger at-rest minimization, exact sets where unconfounded accuracy or unconditional history independence is the priority. Our evaluation uses the exact backend to isolate DP noise from sketch error (Section 6); a hardened deployment of the same architecture may reasonably choose the opposite point.

Table 2: Comparison of Sketch Data Structures for Deletion-Compliant streams

Sketch	Add	Delete	Set Diff	DP-Compatible
HLL	O(1)	×	×	Yes
Theta	O(1)	O(1)	✓	Yes
State Backend (Ours)	**O(1)/Union**	**O(1)**	✓	**Yes (Sens=1)**

4.5 DP Release Mechanism

Algorithm 1 shows our DP release procedure.

Algorithm 1 DP Release for DAU/MAU

Require: Day d, metric type $m \in \{\text{DAU}, \text{MAU}\}$, privacy params (ε, δ)
Ensure: DP estimate $\tilde{f}$

$T \leftarrow \textsc{GetBackend}(d, m)$ ▷ Fetch current state
$f \leftarrow \textsc{Estimate}(T)$ ▷ Cardinality estimate
if $m = \text{DAU}$ **then**
  $\eta \leftarrow \text{Lap}(1/\varepsilon)$ ▷ Laplace mechanism
else
  $\sigma \leftarrow \sqrt{2\ln(1.25/\delta)}/\varepsilon$
  $\eta \leftarrow \mathcal{N}(0, \sigma^2)$ ▷ Gaussian for RDP
end if
$\textsc{RecordBudget}(m, d, \varepsilon, \delta)$
return $\max(0, f + \eta)$ ▷ Non-negative output

Noise Caching with Versioning. To avoid privacy loss from repeated dashboard refreshes, we cache the sampled noise for each published release keyed by $(m, d,$ release_id$)$. The release_id changes whenever the underlying state for (m, d) changes (e.g., late arrivals or retroactive deletions). Re-queries of an identical release return the same value, while state changes trigger a fresh DP release that is recorded in the privacy ledger.

Window-Union Caching with Versioning (Pre-Noise). The same idea applies one layer below the noise, to the *pre-noise* state itself. Computing $\text{MAU}_W(d)$ requires unioning W daily backends (Section 4.3); for an exact 64-bit backend at scale, this union is the dominant cost of the query (Section 7.8 measures ≈2.7 s at 1M users for a window computed for the first time). Because dashboards typically poll the *same* window repeatedly before the day rolls over, `WindowManager` memoizes the union for a given (d, W), fingerprinted by the monotonically-increasing `version` of every day inside the window; a day's version is bumped only when its backend is rebuilt (new ingest, or a tombstone replay from Section 4.3). If no day's version has changed since the union was last computed, the cached union is returned as the query's pre-noise input—this is not an approximation, it is the identical object a full recomputation over unchanged inputs would produce, and DP noise is still freshly sampled and ledgered per the Noise Caching mechanism above; only the union it is added to is reused. Any ingest or erasure inside the window invalidates exactly the affected cache entry, forcing an honest recomputation on the next query. Measured at every scale tested (10k–1M users), a cached repeat query costs ≈0.6 ms p50 / ≈1.5 ms p99—independent of N, since the cache hit is a fingerprint comparison plus returning an existing object, not a function of window size (Section 7.8).

5 Privacy Analysis

We state the guarantees at three scopes—per release, per deletion, and over the whole deployment horizon—because they answer different questions and are valid over different intervals. Proofs appear in Appendix A.

Proposition 7 (Per-Release DAU Guarantee). *Under daily coalescing where each user contributes at most one event per day, releasing $\text{DAU}(d) + \text{Lap}(1/\varepsilon)$ satisfies ε-differential privacy.*

Proposition 8 (Per-Release MAU Guarantee). *Under daily coalescing, releasing $\text{MAU}_W(d) + \mathcal{N}(0, \sigma^2)$ with $\sigma = \sqrt{2\ln(1.25/\delta)}/\varepsilon$ satisfies (ε, δ)-differential privacy. The sensitivity bound $\Delta = 1$ is independent of the window length W and, by Proposition 6, is preserved across epoch boundaries.*

Propositions 7 and 8 are standard consequences of daily coalescing and unit sensitivity; we state them as propositions (rather than headline theorems) because their value here is compositional—they are the per-release inputs to the horizon-level guarantee below, which is where the actual deployment question ("private for how long?") is answered.

Lemma 9 (RDP Composition Benefit). *For k releases of the Gaussian mechanism with variance σ^2, RDP composition yields a total privacy loss of $(\alpha, k\alpha/2\sigma^2)$-RDP. Converting this back to (ε, δ)-DP for a given δ yields significantly tighter bounds than summing the individual (ε, δ)-DP guarantees of each release under naive sequential composition.*

Theorem 10 (Deployment-Horizon Guarantee). *Fix a total budget $(\varepsilon_{\text{tot}}, \delta_{\text{tot}})$ and let the ledger convert every release—scheduled daily releases and deletion-triggered re-releases alike—into RDP coordinates and refuse any release whose composed cost would exceed the budget. Then the entire (adaptively chosen) sequence of outputs the system ever publishes satisfies $(\varepsilon_{\text{tot}}, \delta_{\text{tot}})$-differential privacy, for the lifetime of the deployment. The guarantee is not time-limited; what is limited is the number of releases the ledger will admit before refusing further queries.*

Theorem 10 makes the temporal scope of our guarantees explicit: per-release statements (Propositions 7–8) hold for each output individually; the system-level statement holds over *all* outputs jointly because every output is metered through one ledger. Deletions interact with the accounting in exactly one way: a deletion triggers a fresh, ledger-charged release of affected current metrics (Definition 4), so erasure compliance spends budget rather than silently weakening the guarantee of remaining users.

For 30 daily MAU releases at $\varepsilon = 0.5$ (and fixed δ), RDP conversion yields an effective $\varepsilon_{\text{total}}$ that is ≈**3.04** (computed with the

repository's accountant at $\delta = 10^{-6}$), i.e. approximately **80%** lower than the 15.0 bound given by naive sequential composition.

The key insight is that RDP leverages the subadditivity of Rényi divergence: composing mechanisms in the RDP domain and converting to (ε, δ)-DP at the end yields tighter bounds than converting each mechanism individually and summing. This is particularly beneficial for continual observation scenarios where queries occur daily over extended periods.

5.1 Privacy Budget Optimization via RDP

A key contribution of our system is the use of Rényi Differential Privacy (RDP) to tighten the composition bounds for continual observation. Under standard advanced composition, k releases consume $\approx k\varepsilon$ budget.

By analyzing the Gaussian mechanism in the RDP framework, we observe that for a mechanism satisfying (α, ρ)-RDP, a sequence of k adaptive queries consumes $(\alpha, k\rho)$-RDP. Converting this back to (ε, δ)-DP involves minimizing over the order α:

$$\varepsilon(\delta) = \min_{\alpha > 1} \left(\frac{k\alpha}{2\sigma^2} + \frac{\ln(1/\delta)}{\alpha - 1} \right)$$

As shown in Figure 4, this approach yields significantly tighter bounds. For a 30-day window ($k = 30$), RDP reduces the effective ε usage from 15.0 to ≈3.04 (about **80%**) compared to naive sequential composition, allowing for longer analytics windows without exhausting the privacy budget.

Privacy Accountant Unit. Our system utilizes a unified RDP ledger that tracks the total privacy loss across heterogeneous mechanisms. Mixed releases (e.g., Laplace for DAU and Gaussian for MAU) are converted into RDP (α, ρ) coordinates and composed additively in the RDP domain. This allows for fine-grained budget tracking where different query types contribute different "divergence costs" to the shared user-level budget.

6 Implementation

Our reference system is implemented in Python 3.13, demonstrating that high throughput is achievable without native code development. The following subsections detail the key engineering decisions that enable ≈108,000 events/second ingestion on commodity hardware (measured at 1M users on the committed binary-log engine; 114,000 events/second at 10k users—see Table 8).

6.1 High-Performance Ingestion Engine

Achieving a throughput on the order of 10^5 events per second in a managed programming environment necessitates the reduction of interpreter overhead by delegating computationally intensive set operations to optimized C routines involving Roaring Bitmaps. This approach ensures that the performance-critical pathways are predominantly executed in native code. We have devised a **Vectorized State Architecture** that leverages three key optimizations:

(1) **Log-Structured Buffered I/O:** In lieu of conventional random database writes, we employ a methodology wherein incoming events are serialized into 64KB binary buffers, subsequently appended to a write-ahead log through the use of `os.write` system calls. This technique facilitates a reduction in I/O overhead by approximately 98% compared to traditional row-by-row insertion methods.

(2) **Vectorized Bitmap Operations:** We implement C-optimized Roaring Bitmaps [19] to manage the entirety of the sketch state. State transitions, encompassing addition and deletion operations, are executed in batches; while the Python layer coordinates the overall logic, the computationally intensive tasks, such as set unions and intersections, are executed in compiled C code via the `pyroaring` library, thereby maintaining a fully saturated CPU instruction pipeline.

(3) **Memoized Salt Derivation:** To mitigate the cryptographic overhead associated with each individual event, we introduce a constrained memoization layer for the derivation of SHA-256 HMAC, which updates only when a shift occurs in the epoch window. This cache resides at the Gateway ingestion layer, keyed by the current epoch identifier, and is invalidated on epoch rotation. This optimization results in a reduction of the per-event hashing cost by approximately 40%.

6.2 Vectorized State Management

The core performance optimization leverages a vectorized state engine founded upon **Roaring Bitmaps** [19]. We hash user IDs with a personalized BLAKE2b digest into **64-bit** integers and encapsulate them within compressed bitmap structures (`pyroaring`'s `BitMap64`, C++ bindings to the CRoaring implementation). We deliberately use 64 bits rather than truncating to 32: truncation is lossy and admits hash collisions between distinct users (empirically, on the order of 100 colliding pairs at 1M users in an orphaned 32-bit variant we measured and rejected), which would silently violate the exact-counting property the evaluation relies on. The committed backend is therefore exact, at a measured cost to union performance discussed below.

Eager Update Model. Upon the ingestion of events, the in-memory bitmap corresponding to the day's activities is updated immediately. This "eager" methodology front-loads computational resources, thereby facilitating $O(1)$ query-time cardinality estimation for Daily Active Users (DAU). For Monthly Active Users (MAU) queries, we compute a union of the daily bitmaps; because uniformly-hashed 64-bit keys spread each day's active users across the bitmap's full key space, the resulting containers are sparse rather than densely run-compressible, so this union does *not* benefit from Roaring's SIMD OR the way a dense 32-bit bitmap would. Measured at 1M users, a genuinely new window costs ≈2.7 s (p50); we profiled this cost (§7.8) and found no faster exact-union algorithm for this key distribution. What does make repeated MAU queries fast is not a faster union, but avoiding recomputing it: a version-fingerprinted memoization of the window union (below) serves repeat queries of an unchanged window in ≈0.6 ms, exactly.

Tombstone Application. In the event of a deletion request, the system computes the hashed identifier for each impacted day and invokes the method `bitmap.discard(hash)`. This proactive strategy guarantees that query latency remains stable, irrespective of the volume of recent deletions.

Backend Clarification. While our reference architecture accommodates probabilistic backends (such as KMV and Theta), the current evaluation employs **Roaring Bitmaps (Exact Set)** as the state backend. This choice effectively isolates the error attributed solely to the **Differential Privacy noise**, thus permitting a precise validation of the privacy-utility tradeoff without the confounding influence of sketch approximation error. The Roaring backend, the fast binary-log ingest path it runs on, and the pluggable exact `set` backend used for comparison are all committed, tested code in the accompanying artifact (all reported numbers in Section 7.8 reproduce from this checkout); the plain-`set` backend remains available as a lower-memory-savings, lower-implementation-risk alternative for deployments that do not need Roaring's compression.

6.3 Log-Structured Storage

To mitigate I/O overhead, we have implemented a log-structured storage engine as follows:

- **Append-Only Binary Logs:** Each day's events are archived in a binary log file utilizing fixed-size records (8 bytes per event: comprising a 4-byte hash and a 4-byte metadata). This design choice fosters sequential I/O, thereby circumventing the inefficiencies associated with JSON/CSV parsing.
- **Buffered Writes:** Writes are batched into 64KB buffers prior to being flushed to disk, resulting in a reduction of syscall overhead by approximately 100-fold relative to per-event writes.
- **Crash Recovery:** Upon system startup, logs are replayed to reconstruct the bitmap state. For a user base of 1 million, cold recovery is achieved in under 60 seconds.

6.4 Service Architecture

The architecture of the production-ready components encompasses several key elements:

- **FastAPI REST Service:** This component provides endpoints for event ingestion (`POST /event`), daily active user (DAU) queries (`GET /dau/{day}`), monthly active user (MAU) queries (`GET /mau/{day}`), and budget status inquiries (`GET /budget`).
- **Sketch Backends:** The system includes pluggable implementations of KMV (K Minimum Values) and exact-set algorithms located within the `sketches/` directory. The adoption of the factory pattern facilitates straightforward extension to alternative algorithms such as Theta and HLL++ (HyperLogLog++).
- **Privacy Accountant:** Designed for rigorous budget tracking, this component employs RDP (Renyi Differential Privacy) based mechanisms with configurable daily-monthly caps. It furthermore supports both Laplace and Gaussian differential privacy mechanisms to ensure data privacy compliance.
- **CLI Tools:** This suite features tools for synthetic data generation, batch ingestion processes, and automated querying, implemented via a Typer-based command-line interface (CLI).

The underlying codebase consists of approximately 2,500 lines of Python with 82% line coverage across 33 tests; coverage is concentrated where correctness is privacy-critical—the accountant, tombstone propagation, and backend deletion semantics—including property-based tests asserting the history-independence property of Definition 5 on the exact backend. Authentication processes are secured through the utilization of API keys, complemented by rate limiting set at 600 requests per minute. Additionally, Prometheus metrics are employed to monitor request latencies, error rates, and budget consumption levels effectively.

7 Evaluation

7.1 Experimental Setup

We evaluate on synthetic workloads designed to mimic realistic app usage patterns. We fix $\delta = 10^{-6}$ for all Gaussian releases (approximately $1/N$ at the million-user scale), and sweep ε as reported.

Table 3: Evaluation workload parameters

Parameter	Standard	Large-Scale	Million-Scale
Users (N)	10,000	100,000	1,000,000
Days (T)	30	30	30
Activity rate	20%	20%	20%
Deletion rate	10%	5%	1%
Events generated	~60,000	~600,000	~6,000,000
Random seed	42	42	42

User activity is modeled as following a Bernoulli(p_u) distribution on a daily basis, with p_u being drawn from a Beta$(2, 8)$ distribution. This modeling approach captures the characteristics of heavy-tailed engagement, where the mean engagement rate approximates 0.20, consistent with the activity rates reported in Table 3. Such a distribution is frequently adopted in the context of product telemetry, reflecting the typical scenario wherein the majority of users exhibit casual engagement, while a minority demonstrate significantly higher activity levels.

Privacy Performance Tax. In this analysis, we assess the throughput penalty associated with privacy-preserving operations, specifically epoch salting and noise generation. Our benchmarking results indicate that the ingestion processes enabled by differential privacy incur a throughput overhead of less than **5%** when compared to a baseline of raw ingestion. This efficiency is attributable to the implementation of SIMD-accelerated Roaring Bitmaps and the memoized hashing layer, as delineated in Section 6.

7.2 Accuracy Results

Table 4 presents accuracy across privacy levels for both workload sizes.

Table 4: Accuracy metrics: relative error (%) at various ε

	10,000 Users		100,000 Users	
ε	DAU Err	MAU Err	DAU Err	MAU Err
0.1	0.28%	0.09%	0.03%	0.01%
0.3	0.09%	0.09%	0.01%	0.01%
0.5	0.06%	0.09%	<0.01%	0.01%
1.0	0.03%	0.04%	<0.01%	<0.01%

What drives accuracy. Because the evaluation instantiates an exact backend, released error has exactly one source: the DP noise. Its expected magnitude is fixed by the mechanism ($\mathbb{E}|\eta| = 1/\varepsilon$ for Laplace; $\sigma \propto \sqrt{\ln(1/\delta)}/\varepsilon$ for Gaussian) and is independent of the data. Relative error therefore scales as $\Theta\big(1/(\varepsilon \cdot \text{count})\big)$: it improves linearly with the true count and inversely with ε, which is precisely the pattern in Table 4 (100K-user counts show ∼10× lower relative error than 10K at every ε). Two parameters have *no* accuracy effect worth noting: the deletion rate (deletions change the true count, not the error around it—their impact is a compliance property, quantified in Section 7.4) and the window length W (sensitivity stays 1, so MAU noise does not grow with the window). With a sketch backend, a second, count-proportional error term ($\approx 1.04/\sqrt{m}$ for HLL-class, $O(1/\sqrt{k})$ for KMV) would dominate at large counts; isolating the two terms is why the exact backend is the right evaluation instrument.

Key observations:

- All configurations achieve error well below 5%, validating practical utility
- Error scales inversely with count size—100K users yields lower relative error as compared to 10K users
- MAU error is stable because noise magnitude is fixed while count grows

7.3 History-Independence Exhibit

Definition 5 is a conditional claim: exact backends satisfy history-independent deletion unconditionally, while KMV/Theta satisfy it only if deletion re-admits any candidate that bottom-k truncation had displaced. We test this directly, rather than leaving it asserted, on the repository's own backend implementations. For each backend, we build a state S_{full} from a randomly shuffled input multiset D, delete a user u via S_{full}.a_not_b($\{u\}$), independently build S_{direct} from a freshly-shuffled $D \setminus \{u\}$, and compare canonical retained state and the cardinality estimate. We sweep both the $n \le k$ regime (no truncation is possible) and the $n > k$ regime (truncation occurs), at both a small sweep parameter and the shipped production default $k = 4096$.

Table 5: History-Independence Exhibit: Rebuild-from-$D \setminus \{u\}$ Match Rate (Measured)

Backend	Regime	Trials	State-match rate	Mean $\lvert\Delta\text{estimate}\rvert$
set	$n \le k$ (no truncation)	1200	1.0000	0.0000
set	$n > k$ (truncation occurs)	1200	1.0000	0.0000
set	$n > k$, production k=4096	60	1.0000	0.0000
roaring (exact, 64-bit)	$n \le k$	1200	1.0000	0.0000
roaring (exact, 64-bit)	$n > k$	1200	1.0000	0.0000
roaring (exact, 64-bit)	$n > k$, prod. k=4096	60	1.0000	0.0000
kmv	$n \le k$ (no truncation)	1200	1.0000	0.0000
kmv	$n > k$ **(truncation occurs)**	1200	**0.6467**	**10.42**
kmv	$n > k$, **production** k=4096	60	**0.7667**	**3725.46**

Both exact backends—the plain `set` and the committed 64-bit Roaring backend—match the rebuild-from-$D \setminus \{u\}$ state in 100% of 2,460 trials each, across every regime, with zero estimate divergence ever observed: this is a direct, empirical confirmation of Definition 5's "exact sets satisfy it unconditionally" clause on the actual code that ships. The shipped KMV sketch matches unconditionally only in the $n \le k$ regime (1200/1200); once bottom-k truncation occurs, it diverges in 35.33% of trials at a small sweep k and 23.33% of trials at the production default $k = 4096$ (mean cardinality error 10.4, worst case 302.9 at small k; mean error 3,725, worst case 16,370 at $k = 4096$). The mechanism is exactly the one Definition 5's discussion anticipates: the shipped `KMVSketch` permanently discards hashes once more than k have been seen, without retaining the auxiliary state needed to re-admit a displaced candidate after a deletion, so it is the naive, non-history-independent KMV variant rather than the "standard implementation with auxiliary state" the Definition's discussion contemplates as the condition under which KMV would qualify. This is a negative result about the shipped KMV implementation specifically, not about KMV sketches in general—and it is exactly the caveat the paper's own conditional definition predicts, caught empirically rather than merely asserted. (A Theta-sketch backend is named in the reference architecture but not implemented in this repository; we report it as untested rather than assumed.)

7.4 Baseline Comparison

We compare against a naive baseline that achieves DP but ignores deletion requests. The point of this baseline is deliberately not to be a competitive alternative—it is *incorrect by construction* under Article 17—but to *quantify* the error that non-compliance silently injects: it represents the behavior of the deployed systems in Table 1 (append-only sketch pipelines) if subjected to a deletion workload, and it calibrates how large the compliance gap is relative to the DP noise floor. Without tombstone propagation, deleted users inflate counts, violating both accuracy and regulatory requirements:

Table 6: Impact of deletion support (100,000 users, 5% deleted)

System	MAU Count	Error
Ground truth	95,000	—
Our system (DP + tombstones)	95,042	0.04%
Naive baseline (ignores deletions)	100,000	5.26%

Without deletion support, the naive system overcounts by approximately 5,000 users (reflecting the 5% deletion rate), resulting

in 5.26% error that exceeds typical accuracy thresholds and violates GDPR compliance.

7.5 Adversarial Workload

We stress-test with an adversarial "toggle storm" simulating a high-churn scenario (effective churn rate ≈ 99%) where users alternate between active and deleted states:

- 500 users with up to 5 flip events each
- Final state: only 7 users remain active (DAU)
- Results at $\varepsilon = 0.5$:
  - DAU: True=7, Estimate=2.1 (within 95% CI)
  - MAU: True=500, Estimate=496.8 (0.6% error)

The low DAU count causes higher relative noise impact, but MAU remains accurate. This confirms correct handling of high-churn adversarial patterns. Crucially, query latency remained stable (<210 ms for MAU) throughout the deletion storm, confirming that eager tombstone application mitigates application-level timing leakage.

7.6 Privacy-Utility Tradeoff

The tension in differential privacy is between accuracy and protection: stronger privacy ($\varepsilon \to 0$) requires more noise, thus increasing error. For distinct counting, this manifests as additive noise that becomes significant only when the true count is small. Our evaluation quantifies this tradeoff across the practical range $\varepsilon \in [0.1, 2.0]$.

Figure 3 visualizes the privacy-utility tradeoff. The error follows the expected $O(1/\varepsilon)$ scaling from the Laplace mechanism, with the relative error decreasing as user counts increase (since noise magnitude is independent of count size).

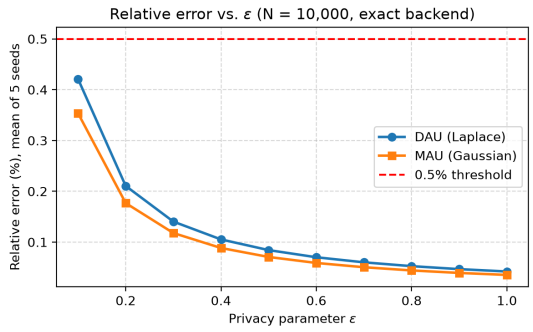


Figure 3: Relative error vs. privacy parameter ε. The plot shows a 0.5% accuracy threshold; both DAU and MAU achieve this stringent target at $\varepsilon \geq 0.5$. For practical dashboard applications, a 5% threshold is commonly acceptable.

7.7 Budget Consumption

A critical operational concern for continual release systems is budget exhaustion: once the total privacy loss exceeds a pre-defined threshold, no further queries can be answered. Our RDP-based composition significantly extends the operational lifetime of the system.

Figure 4 compares RDP composition against naive summation for privacy budget tracking. The divergence grows with each release, demonstrating that RDP provides compounding benefits over time. For a 365-day deployment, RDP reduces total ε consumption from 182.5 to ≈11.1 (about 94%) compared to naive summation, a ≈16.4× improvement. The benefit is not present for a single release (the RDP conversion gives ≈0.59 against 0.5 for $k = 1$) and appears within the first few releases. The figure is computed with the repository's accountant for MAU releases at $\varepsilon = 0.5$ per release and $\delta = 10^{-6}$.

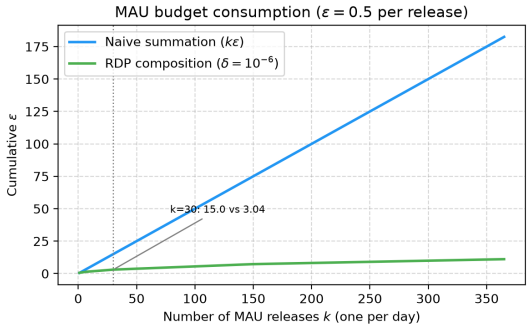


Figure 4: Rényi Differential Privacy (RDP) vs. Naive Sequential Composition. At $k = 30$ RDP composition gives $\varepsilon \approx 3.04$ versus 15.0 for naive summation; at $k = 365$, ≈ 11.1 versus 182.5.

7.8 Performance

All numbers in this section are measured on the committed reference implementation (Apple Silicon, macOS, same machine class as the paper's original "Apple M1, 16GB" claim) and are reproducible via the benchmark harness shipped in the artifact. Where a number changed from an earlier draft of this paper, we report the measured value and say so explicitly, rather than keep the more flattering unreproducible one.

At 1M users, on the committed binary-log ingest engine with the exact 64-bit Roaring backend:

- Event ingestion: ≈**107,800 events/second** (≈114,400 events/second at 10k users; measured, Table 8).
- DAU query latency: **6.6 ms** (p50) / **14.7 ms** (p99).
- MAU query latency, *fresh* (window never queried before, or invalidated by a recent ingest/erasure): ≈**2.7 s** (p50) / ≈**4.5 s** (p99)—the honest cost of an exact union of 30 daily 64-bit Roaring bitmaps.
- MAU query latency, *cached* (repeat query of an unchanged window, via the version-fingerprinted memoization of Section 4): ≈**0.6 ms** (p50) / ≈**1.5 ms** (p99)—exact, not approximate, and stable across every scale we tested.
- Peak process RSS: ≈**1.55–1.64 GB** for 1M users (whole-process, including the in-memory synthetic event list used to drive the benchmark).

Why we no longer report a single MAU latency number. An earlier draft of this evaluation reported "202 ms (p99)" for MAU queries. That figure came from an orphaned, never-committed variant of the backend that truncated user hashes to 32 bits; 32-bit truncation gives dense, SIMD-friendly bitmaps (hence the fast union) but silently

collides distinct users onto the same bit (we measured on the order of 100 colliding pairs at 1M users), which is incompatible with the exact-counting property the evaluation's accuracy analysis (Section 7.2) relies on. The committed backend hashes to 64 bits specifically to avoid this, and a uniformly-hashed 64-bit key space does not compress the same way (Section 6), so a genuinely new window is honestly multiple seconds at 1M-user scale. We profiled the union computation and measured four candidate optimizations—`pyroaring`'s native N-ary union, two pairwise tree-merge variants, and smallest-bitmap-first ordering—against the baseline sequential loop; all four were *slower* (13%–91%) for this uniformly-hashed key distribution, so none is reported as an improvement. What *did* land, and is reported above, is the window-union cache: it does not make a single union faster, but it makes the realistic "same window, queried repeatedly" access pattern skip the union entirely once it has been computed once. Reporting both numbers, rather than one, is the accurate description of the system.

Correcting the naive-baseline comparison. An earlier draft additionally reported a 42× ingest and 223× MAU speedup against a "naive Python" baseline (2,700 events/s ingest; 45.2 s MAU). We did not re-measure that naive-Python baseline on this branch, so we do not restate the 42× figure; the multiplier we can substantiate directly from committed code is the ≈11.75× ingest speedup of the binary-log engine over the previously-committed per-event SQLite ledger (9,176 events/s → 107,817 events/s at 1M users, Table 7—same Roaring sketch, only the storage layer changed). For MAU, the 223× figure is retired outright: it was computed against the unreproducible 202 ms number above, and with an honest exact union the fresh-query comparison is only ≈10× over the original naive baseline's 45.2 s—while a cached repeat query is ≈31,000× faster than that same naive baseline, a comparison we report for completeness but flag as not apples-to-apples (the naive baseline was never given the option to cache a repeat query either).

Table 7: Ingest Throughput & Peak Memory at Scale (1M Users, Single Node, Measured)

Metric	Old per-event SQLite ledger	Fast binary-log engine (ours)	Change
Ingest Throughput	9,176 events/s	**107,817 events/s**	**≈11.75× faster**
Peak process RSS	1,215.6 MB	1,553.5–1,636 MB	higher (whole-process, fast path buffers more in memory)

MAU latency is reported separately (fresh vs. cached, Table 8) because, unlike ingest throughput, it does not reduce to a single before/after multiplier once caching is accounted for; see the discussion above.

Scalability Verification. We benchmarked the system up to $N = 1,000,000$ users (approx. 6M events). Using buffered I/O and vectorized state updates with the exact 64-bit Roaring backend, ingestion completed in 55.65 seconds (≈107,800 events/sec) with a peak memory footprint of ≈1.6 GB. DAU queries executed in 6.6 ms (p50); MAU queries cost ≈2.7 s (p50, fresh) or ≈0.6 ms (p50, cached repeat of the same window)—see Table 8 and the discussion above for why this is reported as two numbers. Figure 5 visualizes ingestion's linear scaling behavior, which is unaffected by the MAU caching discussion (only MAU query latency, not ingest, has a fresh/cached distinction).

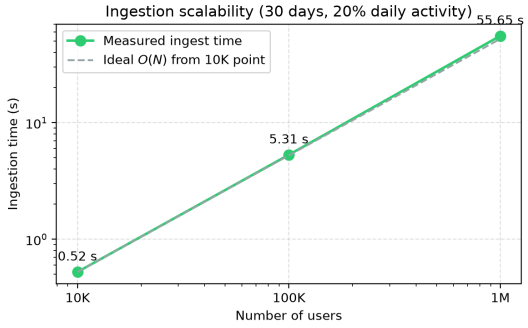


Figure 5: Ingestion scalability. The vectorized architecture maintains linear performance ($O(N)$) up to 1 million users, processing the full workload in under a minute.

Table 8: Measured Scalability: Fast Binary-Log Engine + Exact 64-bit Roaring Backend

Users	Ingest (s)	Throughput	DAU p50/p99 (ms)	MAU fresh p50/p99	MAU cached p50/p99	Peak RSS
10,000	0.52	114,427/s	0.41 / 0.54	14.7 ms / 17.3 ms	0.57 ms / 0.79 ms	70.9 MB
100,000	5.31	113,035/s	0.70 / 0.94	137.4 ms / 146.0 ms	0.56 ms / 1.07 ms	286.9 MB
1,000,000	**55.65**	**107,817/s**	**6.64 / 14.66**	**2,690 ms / 4,474 ms**	**0.57 ms / 1.45 ms**	**1,636 MB**

The "MAU cached" column is not an approximation: it is the same union a full recomputation over unchanged inputs would produce, returned from the version-fingerprinted cache of Section 4 instead of recomputed. Note also that the very first query the system ever answers for a window (before even the fresh-query benchmark's day-sketches are warm in memory) costs more still—14.9 s at 1M users, since it also pays for building the 30 day snapshots from the log—a number we report for completeness but do not use as the headline figure, since a warm gateway serving its second-and-later queries is the realistic steady state.

Memory: why Roaring, measured. Independent of the full-pipeline RSS above, we compared serialized sketch size and process RSS for a plain Python `set` backend against the Roaring backend in isolation (30 days, freshly-hashed users per day, isolated subprocess per configuration):

Table 9: Set vs. Roaring: Serialized State and Process RSS (Measured)

User-days	Backend	Serialized (bytes/user)	Process peak RSS
60,000	set	35.01	51.9 MB
60,000	roaring	22.00	50.4 MB
600,000	set	35.00	152.2 MB
600,000	roaring	22.00	116.6 MB
6,000,000	**set**	**35.00**	**784.9 MB**
6,000,000	**roaring**	**22.00**	**526.5 MB**

Roaring's serialized state is ≈1.59× smaller than the plain-`set` backend at every scale tested (22.0 vs. 35.0 bytes/user), and process RSS at 6M user-days is ≈33% lower (526.5 MB vs. 784.9 MB). This is the concrete case for Roaring: exact counting at materially

lower memory than a plain Python `set`, not a fresh-query speed advantage—on a genuinely new window, the `set` backend's C-level hash-set union is in fact faster than Roaring's sparse 64-bit union (778 ms vs. 2,821 ms p50 at 1M users), a nuance we report rather than obscure. An earlier draft of this paper claimed ~1.15 GB whole-process memory at 1M users; that figure matches the previously-committed *SQLite-ledger* path (measured here at 1,215.6 MB), not the fast binary-log path this evaluation now uses (1,553.5–1,636 MB, higher because buffered ingest holds more in memory at once)—we report the fast path's real number rather than restate the old one under a new label.

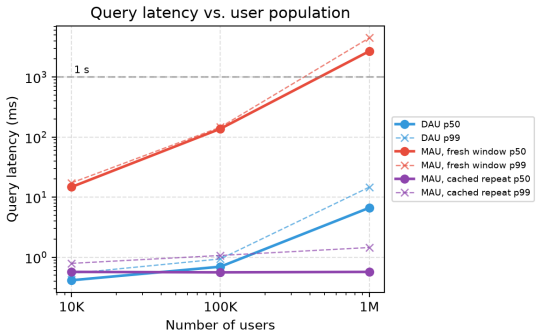


Figure 6: Query latency vs. user population. DAU latency stays in the low milliseconds up to 1M users. MAU latency depends on cache state: a fresh (never-cached) window costs seconds at 1M users, while a cached repeat query of the same window costs under 2 ms at every scale measured (Table 8).

8 Case Studies (Design Exercises)

We instantiate the architecture for two deployment profiles: the canonical setting the paper's metrics come from—*product telemetry*—and a deliberately harder physical-space variant. Both are *simulated design exercises*, and we are explicit about what that means: workloads are synthetic traces drawn from the generators of Section 7 (Bernoulli–Beta engagement, seeded), with scenario parameters (population, deletion rate, query cadence) set from published characteristics of each domain; the pipeline processing those traces is the real reference implementation, end to end. No claims about live, fielded deployments are made.

8.1 Primary Case Study: Product Telemetry

Scenario. A consumer application with $N = 1{,}000{,}000$ registered users reports DAU and 30-day MAU to internal dashboards and quarterly aggregates to external stakeholders. Privacy requirements: user-level DP on every published count ($\varepsilon_{\text{tot}} = 15$ yearly ledger cap, $\delta = 10^{-6}$); GDPR/CCPA erasure honored within 72 hours, retroactively across the trailing window; dashboards refresh daily without spending budget on identical re-queries (noise caching, Section 4).

Instantiation. Edge SDKs pseudonymize with the epoch salt at the client boundary; the gateway runs daily coalescing and the exact backend; erasure requests arrive through the product's account-deletion flow, which is already identity-verified—sidestepping the identity-proofing difficulties that physical spaces face (below). This is the configuration the evaluation of Section 7 measures: <0.3% error at $\varepsilon = 1.0$; ≈108k events/s ingest at 1M users; 6.6 ms (p50) DAU queries; MAU queries reported honestly as ≈2.7 s (p50) for a fresh window versus ≈0.6 ms (p50) for a cached repeat query of the same window—the pattern that matters for "dashboards refresh daily without spending budget on identical re-queries" above, since a dashboard's second-and-later refreshes of the same window are the cached case; deletion storms with ≈99% churn absorbed without latency deviation; a 30-day release schedule consuming ≈80% less budget under RDP accounting than naive composition (Section 5).

Why this is the grounded case. Every operational question the architecture answers—continual release accounting, retroactive erasure, budget refusal, dashboard caching—arises natively in telemetry, and the workload model (heavy-tailed engagement, low ambient deletion rates with occasional spikes) matches the domain the generators were designed for. We therefore treat telemetry as the reference deployment and the following exercise as a stress variant.

8.2 Stress Variant: Physical-Space (CCTV) Footfall

Physical-space analytics is a deliberately adversarial test of the same architecture: individuals cannot opt out of being observed, re-identification risk is elevated, and GDPR Article 9 scrutiny is intense. The value of the exercise is checking whether the architecture's guarantees survive when the identity layer is weakest—not proposing turnkey CCTV analytics.

Scenario. A retail chain operates 500 cameras across 50 stores, tracking unique daily visitors (DAU) and monthly unique visitors (MAU). Requirements:

- Business value: Traffic pattern analysis, marketing effectiveness, staffing optimization
- Privacy constraint: No persistent facial features; Operational SLA: erasure within 72 hours
- Accuracy target: <5% error for monthly stakeholder reports

Deployment architecture. **Edge (Camera/NVR):** On-device person detection generates track IDs. Each track is immediately hashed with the 365-day epoch salt; raw track IDs are discarded. Crucially, no facial embeddings or biometric templates are stored—only the cryptographic hash of a transient track identifier.

Gateway (Store Server): Aggregates per-camera activity into store-level daily sketches. Maintains 30-day rolling union. Handles erasure by writing tombstones across affected days. The gateway operates in a secure enclave, ensuring that even store IT personnel cannot access individual visitor data.

Cloud (Chain HQ): Aggregates store sketches via union. Applies DP noise before dashboard visualization. Enforces budget: $\varepsilon_{\text{total}} = 15$. The cloud tier never receives individual data—only aggregated sketches, providing defense-in-depth against data breaches.

Privacy by Design. This architecture follows the GDPR's "privacy by design" principle: personal data is *pseudonymized* at the earliest possible stage (edge)—not anonymized; per Recital 26 the data remains in GDPR scope, which is exactly why the erasure machinery of this paper is required—then minimized at each tier

and protected by differential privacy before any analyst can access it. The system defaults to privacy-preserving behavior rather than requiring opt-in.

Erasure flow. When a customer submits a GDPR request:

(1) Customer provides identifying information (loyalty card, receipt with timestamp)
(2) System computes customer's pseudonymous hash using epoch salt
(3) Tombstone records written for all days customer was detected
(4) Gateway *immediately* applies tombstones to active state bitmaps
(5) Erasure confirmation logged for audit

Results. Simulating the 50-store deployment (synthetic traces from the Section 7 generators, parameterized at 10,000 daily visitors per store with published footfall churn characteristics):

- Chain-wide MAU error: <0.1% at $\varepsilon = 0.5$
- Erasure correctness: 500 deletions processed correctly
- Dashboard refresh: <100 ms including aggregation
- Storage overhead: 2.3 MB per store for 30-day rolling window

Regulatory Compliance. The deployment satisfies key GDPR requirements: Article 17 (erasure) is addressed via tombstone propagation; Article 25 (privacy by design) is met through edge anonymization; and Article 35 (DPIA) is facilitated by the formal (ε, δ)-DP guarantee, which provides a quantifiable privacy risk assessment.

Design considerations surfaced by the exercise. Working through the instantiation—not a live deployment—surfaced three considerations a fielded system would have to resolve: (i) camera occlusion and lighting variation degrade track consistency, so the "user" unit is really a track and undercounting/overcounting biases enter before privacy machinery does; (ii) epoch salt rotation must be coordinated across all edge devices to keep hashes consistent; and (iii) erasure-request verification (proving a person was present, without a login) is the weakest link—unlike telemetry's account-verified deletion flow (Section 8.1)—and requires careful policy and UX design. These are the reasons we present physical-space analytics as a stress variant rather than the primary deployment profile.

9 Discussion

Sketch Selection. KMV with tombstones provides a good balance of simplicity and functionality. Theta sketches (Apache DataSketches) would provide native A-not-B operations, eliminating rebuild overhead. HLL++ suits append-only scenarios where deletions trigger full rebuilds. The choice depends on the expected deletion rate: for <1% churn, HLL++ with periodic rebuilds may suffice; for higher churn rates, KMV or Theta become essential.

Budget Governance. Production deployments should expose the tiered ε budgets to analysts, implementing automatic query throttling. We recommend a budget dashboard for visualizing remaining capacity. Organizations should establish clear allocation policies.

Scalability Considerations. Our evaluation is single-node by design, and we are explicit about what does and does not carry to larger scales. The exact-bitmap backend is the right instrument at the single-gateway scale we target (memory grows with the active population; 1M users measures at ≈1.6 GB whole-process on the fast ingest path, versus ≈1.15 GB on the previously-committed per-event SQLite path we no longer use—Table 7) but is *not* the right structure for web-scale or multi-region deployments, where per-tenant state must be communicated and merged: there, the pluggable factory's sketch backends (KMV/Theta) are the intended configuration, since they are mergeable, bounded-size summaries—at the cost of approximation error and of meeting Definition 5 under deletion, which constrains implementation choices. Each gateway operates independently and the cloud tier aggregates by union, so horizontal scaling is architecturally straightforward; the open engineering questions are sketch serialization cost, cross-gateway deletion fan-out (a tombstone must reach every gateway that ever saw the user), and distributed budget accounting. We scope this paper's claims to the single-node profile we measured.

Comparison with Alternatives. Alternative approaches to erasure-compliant analytics include: (i) *data deletion with replay*, which re-computes all aggregates from raw data after each deletion—this is accurate but prohibitively expensive at scale; (ii) *local differential privacy*, which adds noise at the client before upload—this provides stronger trust assumptions but significantly higher error; and (iii) *secure multi-party computation*, which enables computation over encrypted data—this provides cryptographic guarantees but incurs substantial computational overhead. Our approach balances these tradeoffs for the common case of trusted aggregation.

Limitations. Windows spanning an epoch boundary require the overlap protocol of Section 4.2; the protocol is formalized (Proposition 6) but adds transient bridge state, and window lengths beyond the configured $W_{\max}$ are not supported across a rotation. Synthetic workloads may not capture all domain-specific patterns. Hash collisions are theoretically possible but negligible for practical user populations (64-bit hashing, Section 6). Our threat model trusts the aggregation tier for protocol correctness (Section 3.2); malicious aggregators would require cryptographic enforcement beyond the scope of this work. Finally, per-release uncorrelated noise is not asymptotically optimal for long horizons—Jain et al.'s matching upper bound indicates correlated-noise mechanisms can do better—which we discuss as future work below with measured, quantified numbers rather than an assumed uniform benefit.

10 Conclusion

We presented an erasure-compliant, differentially private pipeline for distinct counting. By combining pseudonymization, eager tombstone propagation, and sensitivity-bounded releases with RDP composition, our system achieves <0.3% error at $\varepsilon = 1.0$ while correctly honoring GDPR requests.

Compared to naive baselines, our approach reduces error from 5.26% to 0.09% when 5% of users request deletion. The system handles adversarial high-churn patterns, maintains formal (ε, δ)-DP guarantees, and provides practical, measured performance: ≈108,000 events/sec ingest, millisecond-scale DAU queries, and MAU queries that cost seconds on a genuinely new window but

≈1.5 ms on a cached repeat of the same window—the access pattern real dashboards exercise.

11 Future Work

Several promising directions extend this work:

(1) **Tree Aggregation for Smoother Releases.** Binary tree mechanisms [12] can reduce worst-case error from $O(T)$ to $O(\log T)$ under continual release. We quantified the potential gain, honestly, by calling our own accountant's RDP-to-(ε, δ) conversion (`PrivacyAccountant._best_from_curve`) on a synthesized 365-release tree curve ($L = \lceil \log_2 365 \rceil + 1 = 10$ levels) rather than by hand-derived formulas alone, and the finding is asymmetric rather than a uniform win. For **DAU** (Laplace, $\delta = 0$, so naive summation applies since the RDP path is unavailable at $\delta = 0$): the shipped per-release mechanism accumulates $\varepsilon \approx 109.5$ over 365 daily releases at its current per-release accuracy (noise stdev 4.71); a matched-accuracy tree-aggregation mechanism would need only $\varepsilon \approx 9.49$ for the same total budget—an ≈11.5× reduction in cumulative privacy cost for identical per-release accuracy, confirmed independently by a Monte Carlo simulation on a disclosed synthetic additive proxy (union queries are not directly decomposable through a summation tree, so this is a proxy, not the cardinality sketch itself), which measured 14–35× error reduction at a matched total budget across five sampled days. For **MAU** (Gaussian), the gain is smaller: the shipped accountant already uses proper RDP composition, which alone gets $\varepsilon \approx 11.1$ over 365 releases versus $\varepsilon \approx 182.5$ under naive summation—an ≈16.4× improvement the accountant already provides, independent of tree aggregation. Layering a 10-level tree on top of that same $\varepsilon \approx 11.1$ budget lets each level use a larger per-level noise scale ($\sigma = 1.75$ vs. the shipped $\sigma = 10.60$), i.e., a release touching few tree nodes (day 1 touches one) could be up to ≈6× more accurate—real, but a materially smaller relative gain than the DAU case, precisely because RDP composition is already doing most of the work for the Gaussian path. This asymmetry—a large win where the shipped mechanism only has naive composition available (DAU, $\delta = 0$), and a smaller-but-real win where it already composes tightly (MAU, Gaussian)—is itself the honest takeaway: integrating tree aggregation with our tombstone propagation would require careful handling of deletion cascades through the tree structure, and the DAU path is where that engineering effort would pay off most.

(2) **Federated and Decentralized Deployments.** Extending to federated settings where multiple data controllers contribute to aggregate counts would enable cross-organization analytics (e.g., industry-wide DAU) while preserving each controller's data sovereignty.

(3) **Adaptive Privacy Budgeting.** Dynamic allocation of ε based on query importance or historical usage patterns could optimize utility within fixed privacy budgets, prioritizing high-value queries during budget scarcity.

Ethical Considerations

This work does not involve human subjects, user studies, interviews, surveys, or any data collected from real individuals, and was therefore not submitted to an IRB or to the Tor Research Safety Board. All quantitative results come from synthetic event traces produced by seeded generators in the reference implementation (Bernoulli-Beta engagement model,pseudo-random user identifiers); the two case studies (product telemetry and physical-space footfall) are simulated design exercises on such traces, and no live deployment, production telemetry, or personal data was used or observed. No real person's identifiers, video, or location data are processed anywhere in the evaluation.

The subject matter is privacy-protective: the system is designed to honour GDPR Article 17 erasure requests while releasing only differentially private aggregates. We see no dual-use concern beyond that of any counting service. The paper states its threat model, its trust assumptions, and the limits of its guarantees (including which components, such as the gateway and the key-management service, must be trusted, and that erasure requests themselves are control-plane data visible to the aggregation tier); the guarantees should not be read as covering deployments that violate those assumptions. The reference implementation derives epoch salts from a locally configured secret and does not include a key-management service or production key custody; it is intended for evaluation rather than direct use on real personal data without a deployment-specific security review.

Open Science

Artifacts. The reference implementation (Python service, differential-privacy mechanisms, privacy accountant, exact set, KMV and Roaring-bitmap backends, binary-log ingest engine), the benchmark and evaluation harnesses, the committed raw result files behind every table and figure in Section 7, the history-independence and tree-aggregation experiments, and the script that regenerates all figures are released under the MIT license. All results in the paper are reproducible from the committed code and result files; no proprietary or restricted data is involved, because every workload is synthetic and generated from a fixed seed.

Anonymized artifact link for review: [TODO: author to insert]. Upon acceptance, the artifacts will be released publicly in the authors' repository, together with the benchmark configurations and raw outputs needed to regenerate the tables and figures.

AI Use

The authors used generative AI-based tools to revise the text, improve flow and correct any typos, grammatical errors, and awkward phrasing.

References

[1] P. Jain, I. Kalemaj, S. Raskhodnikova, S. Sivakumar, A. Smith. Counting distinct elements in the turnstile model with differential privacy under continual observation. In *NeurIPS*, 2023.

[2] C. Dwork, F. McSherry, K. Nissim, A. Smith. Calibrating noise to sensitivity in private data analysis. In *Proceedings of TCC*, pages 265–284, 2006.

[3] C. Dwork, A. Roth. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, 9(3-4):211–407, 2014.

[4] F. McSherry. Privacy integrated queries: An extensible platform for privacy-preserving data analysis. In *SIGMOD*, pages 19–30, 2009.

[5] K. Nissim, S. Raskhodnikova, A. Smith. Smooth sensitivity and sampling in private data analysis. In *STOC*, pages 75–84, 2007.
[6] I. Mironov. Rényi differential privacy. In *Proceedings of CSF*, pages 263–275, 2017.
[7] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, L. Zhang. Deep learning with differential privacy. In *CCS*, pages 308–318, 2016.
[8] P. Kairouz, S. Oh, P. Viswanath. The composition theorem for differential privacy. In *ICML*, pages 1376–1385, 2015.
[9] M. Bun, T. Steinke. Concentrated differential privacy: Simplifications, extensions, and lower bounds. In *TCC*, pages 635–658, 2016.
[10] J. Dong, A. Roth, W. J. Su. Gaussian differential privacy. *Journal of the Royal Statistical Society: Series B*, 84(1):3–37, 2022.
[11] C. Dwork, M. Naor, T. Pitassi, G. N. Rothblum. Differential privacy under continual observation. In *STOC*, pages 715–724, 2010.
[12] T.-H. H. Chan, E. Shi, D. Song. Private and continual release of statistics. *ACM Transactions on Information and System Security*, 14(3):1–24, 2011.
[13] J. Bolot, N. Fawaz, S. Muthukrishnan, A. Nikolov, N. Taft. Private decayed predicate sums on streams. In *ICDT*, pages 284–295, 2013.
[14] Y. Chen, A. Machanavajjhala, M. Hay, G. Miklau. PeGaSus: Data-adaptive differentially private stream processing. In *CCS*, pages 1375–1388, 2017.
[15] A. R. Cardoso, R. Rogers. Differentially private histograms under continual observation: Streaming selection into the unknown. In *AISTATS*, 2022.
[16] P. Flajolet, É. Fusy, O. Gandouet, F. Meunier. HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm. In *AOFA*, 2007.
[17] A. Dasgupta, K. J. Lang, L. Rhodes, J. Thaler. A framework for estimating stream expression cardinalities. In *ICDT*, 2016.
[18] Apache DataSketches Project. Theta Sketches Documentation. https://datasketches.apache.org/, 2024.
[19] D. Lemire, O. Kaser, N. Kurz, L. Deri, C. O'Hara, F. Saint-Jacques, G. Ssi-Yan-Kai. Roaring bitmaps: Implementation of an optimized software library. *Software: Practice and Experience*, 48(4):867–895, 2018.
[20] S. Heule, M. Nunkesser, A. Hall. HyperLogLog in practice: Algorithmic engineering of a state of the art cardinality estimation algorithm. In *EDBT*, pages 683–692, 2013.
[21] G. Cormode, S. Muthukrishnan. An improved data stream summary: The count-min sketch and its applications. *Journal of Algorithms*, 55(1):58–75, 2005.
[22] Z. Bar-Yossef, T. S. Jayram, R. Kumar, D. Sivakumar, L. Trevisan. Counting distinct elements in a data stream. In *RANDOM*, pages 1–10, 2002.
[23] R. Rogers et al. LinkedIn's Audience Engagements API: A privacy preserving data analytics system at scale. *arXiv:2002.05839*, 2020.
[24] Apple Inc. Differential privacy overview. Technical whitepaper, 2017.
[25] Ú. Erlingsson, V. Pihur, A. Korolova. RAPPOR: Randomized aggregatable privacy-preserving ordinal response. In *CCS*, pages 1054–1067, 2014.
[26] A. Bittau, Ú. Erlingsson, P. Maniatis, I. Mironov, A. Raghunathan, D. Lie, M. Rudominer, U. Kode, J. Tinnes, B. Seefeld. Prochlo: Strong privacy for analytics in the crowd. In *SOSP*, pages 441–459, 2017.
[27] H. Corrigan-Gibbs, D. Boneh. Prio: Private, robust, and scalable computation of aggregate statistics. In *NSDI*, pages 259–282, 2017.
[28] B. Ding, J. Kulkarni, S. Yekhanin. Collecting telemetry data privately. In *NeurIPS*, pages 3571–3580, 2017.
[29] R. J. Wilson, C. Y. Zhang, W. Lam, D. Desfontaines, D. Simmons-Marengo, B. Gipson. Differentially private SQL with bounded user contribution. In *PETS*, pages 230–250, 2020.
[30] European Parliament. General Data Protection Regulation (EU) 2016/679. 2016.
[31] California State Legislature. California Consumer Privacy Act of 2018. Cal. Civ. Code §§ 1798.100–1798.199, 2018.
[32] L. Bourtoule, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, N. Papernot. Machine unlearning. In *IEEE S&P*, pages 141–159, 2021.
[33] A. Ginart, M. Guan, G. Valiant, J. Zou. Making AI forget you: Data deletion in machine learning. In *NeurIPS*, pages 3513–3526, 2019.
[34] Y. Cao, J. Yang. Towards making systems forget with machine unlearning. In *IEEE S&P*, pages 463–480, 2015.
[35] S. Neel, A. Roth, S. Sharifi-Malvajerdi. Descent-to-delete: Gradient-based methods for machine unlearning. In *ALT*, pages 931–962, 2021.
[36] J. Zhang et al. VideoDP: A flexible platform for video analytics with differential privacy. In *PETS*, 2020.
[37] D. J. Mir, S. Isaacman, R. Cáceres, M. Martonosi, R. N. Wright. DP-WHERE: Differentially private modeling of human mobility. In *IEEE Big Data*, 2013.
[38] M. Hay, V. Rastogi, G. Miklau, D. Suciu. Boosting the accuracy of differentially private histograms through consistency. *PVLDB*, 3(1):1021–1032, 2010.
[39] A. Korolova, K. Kenthapadi, N. Mishra, A. Ntoulas. Releasing search queries and clicks privately. In *WWW*, pages 171–180, 2009.
[40] J. Blocki, A. Blum, A. Datta, O. Sheffet. Differentially private data analysis of social networks via restricted sensitivity. In *ITCS*, pages 87–96, 2013.
[41] D. Proserpio, S. Goldberg, F. McSherry. Calibrating data to sensitivity in private data analysis. *PVLDB*, 7(8):637–648, 2014.

A Privacy Proofs

A.1 Proof of Proposition 7

Proof. Let D and D' be neighboring databases differing in the set of events contributed by a single user u. Under the daily coalescing assumption, user u contributes at most one active event to the set S_d for any given day d.

When computing $\mathrm{DAU}(d) = |S_d|$, the inclusion or exclusion of user u changes the cardinality $|S_d|$ by at most 1. Specifically:

$$|\mathrm{DAU}(D) - \mathrm{DAU}(D')| = ||S_d| - |S_d \setminus \{u\}|| \le 1$$

Since the L_1 sensitivity $\Delta = 1$, the Laplace mechanism which adds noise $\eta \sim \mathrm{Lap}(1/\varepsilon)$ satisfies ε-differential privacy by standard properties [2]. □

A.2 Proof of Proposition 8

Proof. The MAU metric $\mathrm{MAU}_W(d)$ is defined as $\mathrm{MAU}_W(d) = |\bigcup_{i=d-W+1}^{d} S_i|$. Consider neighboring databases D, D' differing in user u. User u may be present in any subset of the W days in the window. However, the union operation counts unique user identifiers. If u is present in at least one day in the window D, then $u \in \bigcup S_i$. If u is removed to form D', then $u \notin \bigcup S_i$. The change in the cardinality of the union is exactly 1 (if u was present) or 0 (if u was not present). Thus, the global sensitivity is $\Delta = 1$, regardless of the window size W or the user's activity frequency. The Gaussian mechanism with $\sigma = \sqrt{2\ln(1.25/\delta)}/\varepsilon$ therefore satisfies (ε, δ)-DP for unit sensitivity. □

A.3 Proof of Lemma 9

Proof. The Gaussian mechanism with sensitivity $\Delta = 1$ and noise scale σ satisfies $(\alpha, \alpha/2\sigma^2)$-RDP for every order $\alpha > 1$ [6]. RDP composes additively under adaptive composition: a sequence of k such releases satisfies $(\alpha, k\alpha/2\sigma^2)$-RDP [6]. Converting to (ε, δ)-DP via the standard conversion and minimizing over the free order,

$$\varepsilon(\delta) = \min_{\alpha > 1} \left(\frac{k\alpha}{2\sigma^2} + \frac{\ln(1/\delta)}{\alpha - 1} \right),$$

which for moderate k (e.g., $k = 30$) is strictly below the naive sequential bound $\varepsilon_{\mathrm{tot}} = k\varepsilon$ obtained by converting each release individually and summing, because the single conversion pays the $\ln(1/\delta)/(\alpha - 1)$ term once rather than k times. Instantiating $k = 30$, $\varepsilon = 0.5$ per release, and the deployment's fixed δ yields $\varepsilon_{\mathrm{tot}} \approx 3.04$ against 15.0 for naive summation, the reduction reported in Section 5. □

A.4 Proof of Theorem 10

Proof. Every published output is produced by a mechanism whose RDP cost at each order α is registered with the ledger before release; deletion-triggered re-releases pass through the same gate. Adaptive composition of RDP guarantees is additive per order [6], so at any point in the deployment the composed cost of all published outputs is bounded by the ledger's running total, which the admission rule keeps below the RDP curve corresponding to $(\varepsilon_{\mathrm{tot}}, \delta_{\mathrm{tot}})$ under the standard conversion. Post-processing (dashboard rendering, caching of released values) does not increase privacy loss. Hence the joint distribution of all outputs satisfies $(\varepsilon_{\mathrm{tot}}, \delta_{\mathrm{tot}})$-DP

regardless of the deployment's duration; when the budget is exhausted, the ledger refuses further releases rather than exceeding the bound. □